SNOWFLAKE DATA ENGINEERING PROJECT

Healthcare Intelligence Hub

Patient, Cost & Care Analytics

Phase 6 — Performance Testing

Institutional template-based phase document

Student	Aashi Bindal
Enrollment No.	0827CI241004
Team ID	[Team ID]
Institution	Acropolis Institute of Technology and Research · Department of CSIT
Faculty Guide	[Faculty Guide]
Academic Session	[Academic Session]
Document date	October 2026

Prepared from the verified project source (query.sql, Health/streamlit_app.py, Health/snowflake.yml, six CSV datasets). Bracketed fields are placeholders for information not supplied.

Contents

6.1 Project Performance Testing (template) 1

6.2 Testing Objectives and Scope 2

6.3 Test Strategy and Methodology 2

6.4 Executed Test Cases (local replica) 3

6.4.1 Data completeness, uniqueness, validity and referential integrity (summary) 5

6.4.2 SQL transformation validation 5

6.5 Snowflake Test Cases (Not Executed) 5

6.6 Dashboard Functional and Filter Tests (Not Executed, manual) 6

6.7 Query Performance and Resource Usage 7

6.8 Defects, Limitations and Corrective Actions 7

Phase 6 — Performance Testing

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Template covered:** Project Development Phase — Performance Test (uploaded as `Tableau_.docx`)

> **Honesty statement.** Only the tests marked **Executed (local replica)** were actually run. They were run on 21 October 2026 with `tests/local_validation.py`, which rebuilds the `query.sql` logic in an in-memory SQLite database from the six CSVs. No Snowflake account was available during documentation, so every Snowflake and live-dashboard test is marked **Not Executed**, with instructions to run it. No Snowflake timings or pass rates are claimed.

6.1 Project Performance Testing (template)

Field	Value
Date	10 February 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	[blank on template]

Project team shall fill the following information in the model performance testing template.

Table 6.1 — Performance test template

S.No.	Parameter	Screenshot / Values
1.	Data Rendered	6 source tables, 14,059 rows: patients 3,000; doctors 60; facilities 10; diagnoses 15; date 974; patient_visits 10,000. The app renders all 10,000 fact rows (32 columns) when no filter is set. The footer reads "Data filtered to 10,000 visits". *[Insert screenshot of the unfiltered app.]*
2.	Data Preprocessing	RAW: typed load with `CSV_FORMAT` (NULL_IF, date format). STAGING: `TRIM` on all text, `INITCAP` on 10 categorical columns, `WHERE key IS NOT NULL` on 6 tables. CURATED: `SELECT DISTINCT` dimensions; fact built with 4 `LEFT JOIN` s. App: upper-case column names, `pd.to_datetime(VISIT_DATE)` . Verified result: 0 rows dropped, 0 NULL or duplicate keys, 0 orphans.
3.	Utilization of Filters	5 sidebar filters: Date Range, Facility (10 + All), Doctor Specialty (10 + All), Visit Type (5 + All), Diagnosis Category (10 + All). They combine with AND and drive all 6 KPIs, 8 charts and 2 tables. Filter logic verified locally (TC-APP-FLT-01…03). Live UI test **Not Executed**.
4.	Calculation fields Used	SQL: `ROUND(SUM/AVG …, 2)` , `COUNT(DISTINCT …)` , `CASE WHEN READMISSION_WITHIN_30_DAYS='Yes'` , `READMISSION_RATE_PCT = 100·readmissions/visits` , `DATE_TRUNC('MONTH')` . App (pandas): total visits, unique patients, total cost, avg satisfaction, readmission rate %, avg LOS; monthly period; per-diagnosis readmission %; mean cost components (melted); facility and doctor aggregates.
5.	Dashboard design	**No of Visualizations / Graphs: 8 charts** (monthly trend line; visit-type donut; top-10 diagnoses bar; admission-status donut; facility bar; treatment-status bar; readmission-rate bar; cost-breakdown stacked bar), **plus 2 tables** (facility, doctor leaderboard) **and 6 KPI cards**.
6.	Story Design	**No of Visualizations / Graphs: 0 in Tableau** (no Tableau story in the project). Presentation storyline: 6 story points over the Streamlit visuals (Phase 5, §5.14).

6.2 Testing Objectives and Scope

Objectives: confirm that (1) every source row reaches each layer; (2) keys are complete, unique and referentially intact; (3) transformations behave as designed; (4) KPI views are numerically correct; (5) dashboard filter logic is correct; and (6) performance is acceptable for the data volume.

In scope: `query.sql` (all layers), the six CSVs, the logic in `streamlit_app.py`. **Out of scope:** load testing with concurrent users, security penetration testing, and browser compatibility.

6.3 Test Strategy and Methodology

Table 6.2 — Test levels and environments

Level	What	Environment	Tool	Status
Unit (SQL logic)	Each STAGING, CURATED and MART statement	E1: local replica (SQLite + Python UDF emulating Snowflake `INITCAP`)	`tests/local_validation.py`	**Executed**
Data quality	Completeness, uniqueness, validity, referential integrity, consistency	E1	same	**Executed**
Integration	RAW → STAGING → CURATED → MART row-flow and KPI reconciliation	E1	same + independent pandas recomputation	**Executed**
Functional (dashboard logic)	Filter and KPI calculations	E1 (pandas, same expressions as the app)	same	**Executed**
Platform	Same checks inside Snowflake	E2: Snowflake	`tests/snowflake_validation.sql`	**Not Executed**
Functional (UI)	Widgets, charts and tables in the deployed app	E3: Streamlit in Snowflake	Manual checklist §6.6	**Not Executed**
Performance	Query elapsed time and bytes scanned	E2	`INFORMATION_SCHEMA.QUERY_HISTORY`	**Not Executed** (local timings only)

Replica fidelity: the replica uses the same SQL text as `query.sql`, with two substitutions. `INITCAP` is a Python function implementing Snowflake's documented behaviour (default delimiters include whitespace and `-`; letters after the first are lower-cased). `DATE_TRUNC('MONTH', d)` is replaced by `SUBSTR(d,1,7)`. Snowflake-specific behaviour should still be confirmed with TC-SF-06.

How to run:

```
# from the repository root (Python 3.11+, pandas)
python tests/local_validation.py --json validation_results.json
```

6.4 Executed Test Cases (local replica)

Common preconditions: the six CSVs are in the repository root; Python 3.11+ with pandas is installed.
Common steps: run the command above; the script loads the CSVs into RAW, builds STAGING, CURATED and MART, then evaluates each check.

Table 6.3 — Executed checks (local replica, 21 Oct 2026)

Test ID	Category	Test (check)	Expected	Actual	Status
TC-ING-01	Ingestion	PATIENTS_RAW row count equals CSV data rows	3000	3000	PASS
TC-ING-02	Ingestion	DOCTORS_RAW row count equals CSV data rows	60	60	PASS
TC-ING-03	Ingestion	FACILITIES_RAW row count equals CSV data rows	10	10	PASS
TC-ING-04	Ingestion	DIAGNOSES_RAW row count equals CSV data rows	15	15	PASS
TC-ING-05	Ingestion	DATE_RAW row count equals CSV data rows	974	974	PASS
TC-ING-06	Ingestion	PATIENT_VISITS_RAW row count equals CSV data rows	10000	10000	PASS
TC-DQ-NUL-01	Completeness	PATIENTS.PATIENT_ID has no NULLs	0	0	PASS
TC-DQ-DUP-01	Uniqueness	PATIENTS.PATIENT_ID is unique	0	0	PASS
TC-TRF-CNT-01	Transformation	STAGING PATIENTS keeps all RAW rows (no unintended filtering)	3000	3000	PASS
TC-DQ-NUL-02	Completeness	DOCTORS.DOCTOR_ID has no NULLs	0	0	PASS
TC-DQ-DUP-02	Uniqueness	DOCTORS.DOCTOR_ID is unique	0	0	PASS
TC-TRF-CNT-02	Transformation	STAGING DOCTORS keeps all RAW rows (no unintended filtering)	60	60	PASS
TC-DQ-NUL-03	Completeness	FACILITIES.FACILITY_ID has no NULLs	0	0	PASS
TC-DQ-DUP-03	Uniqueness	FACILITIES.FACILITY_ID is unique	0	0	PASS
TC-TRF-CNT-03	Transformation	STAGING FACILITIES keeps all RAW rows (no unintended filtering)	10	10	PASS
TC-DQ-NUL-04	Completeness	DIAGNOSES.DIAGNOSIS_ID has no NULLs	0	0	PASS
TC-DQ-DUP-04	Uniqueness	DIAGNOSES.DIAGNOSIS_ID is unique	0	0	PASS
TC-TRF-CNT-04	Transformation	STAGING DIAGNOSES keeps all RAW rows (no unintended filtering)	15	15	PASS
TC-DQ-NUL-05	Completeness	DATE_STG.DATE has no NULLs	0	0	PASS
TC-DQ-DUP-05	Uniqueness	DATE_STG.DATE is unique	0	0	PASS
TC-TRF-CNT-05	Transformation	STAGING DATE_STG keeps all RAW rows (no unintended filtering)	974	974	PASS
TC-DQ-NUL-06	Completeness	PATIENT_VISITS.VISIT_ID has no NULLs	0	0	PASS
TC-DQ-DUP-06	Uniqueness	PATIENT_VISITS.VISIT_ID is unique	0	0	PASS
TC-TRF-CNT-06	Transformation	STAGING PATIENT_VISITS keeps all RAW rows (no unintended filtering)	10000	10000	PASS
TC-DQ-RI-01	Referential integrity	Every visit PATIENT_ID matches a dimension row	0	0	PASS

Test ID	Category	Test (check)	Expected	Actual	Status
TC-DQ-RI-02	Referential integrity	Every visit DOCTOR_ID matches a dimension row	0	0	PASS
TC-DQ-RI-03	Referential integrity	Every visit FACILITY_ID matches a dimension row	0	0	PASS
TC-DQ-RI-04	Referential integrity	Every visit DIAGNOSIS_ID matches a dimension row	0	0	PASS
TC-DQ-RI-05	Referential integrity	Every VISIT_DATE exists in the DATE dimension	0	0	PASS
TC-TRF-FACT-01	Transformation	FACT row count equals STAGING visits (LEFT JOINs do not fan out)	10000	10000	PASS
TC-DQ-VAL-01	Validity	TOTAL_COST = CONSULTATION + MEDICATION + LAB + ROOM	0	0	PASS
TC-DQ-VAL-02	Validity	INSURANCE_COVERAGE + PATIENT_PAYMENT = TOTAL_COST (±0.01)	0	0	PASS
TC-DQ-VAL-03	Validity	No negative cost / payment values	0	0	PASS
TC-DQ-VAL-04	Validity	PATIENT_SATISFACTION within 1-5	0	0	PASS
TC-DQ-VAL-05	Validity	LENGTH_OF_STAY_DAYS >= 0	0	0	PASS
TC-DQ-VAL-06	Validity	READMISSION flag is exactly 'Yes' or 'No' after INITCAP	0	0	PASS
TC-DQ-VAL-07	Validity	INSURANCE_COVERAGE never exceeds TOTAL_COST	0	0	PASS
TC-DQ-VAL-08	Validity	DATE_OF_BIRTH earlier than REGISTRATION_DATE	0	0	PASS
TC-DQ-VAL-09	Validity	No visit dated before the patient's birth	0	0	PASS
TC-DQ-BR-01	Business rule	'Self Pay' patients receive no insurance coverage	0	1578	**FAIL**
TC-DQ-BR-02	Business rule	'Admitted' visits have LENGTH_OF_STAY_DAYS >= 1	0	653	**FAIL**
TC-MART-01	Transformation	VW_DOCTOR_ANALYTICS has one row per DOCTOR_ID	60	59	**FAIL**
TC-MART-02	Transformation	Sum of VW_FACILITY_ANALYTICS visits = total visits	10000	10000	PASS
TC-MART-03	Transformation	Sum of VW_MONTHLY_VISITS visits = total visits	10000	10000	PASS
TC-TRF-STD-01	Transformation	INITCAP preserves acronym 'ENT' in SPECIALTY	original labels (incl. `ENT`)	`Ent` produced	**FAIL**
TC-TRF-STD-02	Transformation	INITCAP preserves 'Follow-up' in VISIT_TYPE	original labels (incl. `Follow-up`)	`Follow-Up` produced	**FAIL**
TC-KPI-01	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_VISITS matches independent pandas result	10000	10000	PASS
TC-KPI-02	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_PATIENTS matches independent pandas result	2889	2889	PASS

Test ID	Category	Test (check)	Expected	Actual	Status
TC-KPI-03	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_COST matches independent pandas result	82818797	82818797	PASS
TC-KPI-04	KPI reconciliation	VW_EXECUTIVE_KPI.AVG_PATIENT_SATISFACTION matches independent pandas result	3.83	3.83	PASS
TC-KPI-05	KPI reconciliation	VW_EXECUTIVE_KPI.AVG_LENGTH_OF_STAY matches independent pandas result	0.97	0.97	PASS
TC-KPI-06	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_READMISSIONS matches independent pandas result	1203	1203	PASS
TC-APP-FLT-01	Dashboard logic	Visit Type filter = 'Emergency' row count	1465	1465	PASS
TC-APP-FLT-02	Dashboard logic	Date range 2025-01-01..2025-12-31 row count	3812	3812	PASS
TC-APP-FLT-03	Dashboard logic	Empty filter result yields 0 readmission rate (no divide-by-zero)	0	0	PASS

Summary: 55 checks executed, 50 PASS, 5 FAIL. All structural checks (ingestion, completeness, uniqueness, referential integrity, cost and payer reconciliation, KPI reconciliation, filter logic) pass. The five failures are real findings about the data or the SQL, described in §6.8.

6.4.1 Data completeness, uniqueness, validity and referential integrity (summary)

Dimension	Checks	Result
Completeness (NULL keys)	6 tables	0 NULL keys
Uniqueness (duplicate keys)	6 tables	0 duplicates
Referential integrity	4 FKs + visit date → DATE	0 orphans
Validity (ranges, domains, reconciliations)	9 rules	0 violations
Business-rule plausibility	2 assumed rules	2 violations (data anomalies)

6.4.2 SQL transformation validation

- STAGING keeps 100% of RAW rows in all 6 tables (TC-TRF-CNT-01...06).
- FACT has exactly 10,000 rows: the LEFT JOINs do not fan out (TC-TRF-FACT-01).
- Monthly and facility view totals add back to 10,000 (TC-MART-02/03).
- Six executive KPIs match an independent pandas computation on the raw CSV (TC-KPI-01...06).
- Defects found: INITCAP label changes (TC-TRF-STD-01/02) and doctor-view grain (TC-MART-01).

6.5 Snowflake Test Cases (Not Executed)

Precondition: `query.sql` executed in full and RAW tables loaded. **Steps:** open a worksheet, run the matching block of `tests/snowflake_validation.sql`, and record the output.

Table 6.4 — Snowflake platform tests

Test ID	Test	Expected result	Actual	Status
TC-SF-01	Row counts per layer	10000, 10000, 10000, 3000, 3000, 60, 10, 15, 974	[record]	Not Executed

Test ID	Test	Expected result	Actual	Status
TC-SF-02	Cost reconciliation	0 failures	[record]	Not Executed
TC-SF-03	Payer reconciliation	0 failures	[record]	Not Executed
TC-SF-04	Validity ranges	0 failures	[record]	Not Executed
TC-SF-05	Visit dates in DIM_DATE	0 failures	[record]	Not Executed
TC-SF-06	INITCAP side effects	Replica predicts `Ent , Follow-Up`	[record]	Not Executed
TC-SF-07	Doctor view grain	Replica predicts 59 view rows vs 60 doctors	[record]	Not Executed
TC-SF-08	KPI baseline	10000, 2889, 82818797.00, 3.83, 0.97, 1203	[record]	Not Executed
TC-SF-PERF	Elapsed time per query	Record `ELAPSED_S` , `BYTES_SCANNED`	[record]	Not Executed

6.6 Dashboard Functional and Filter Tests (Not Executed, manual)

Precondition: app deployed in Snowsight and opened by a role with SELECT on `CURATED.FACT_PATIENT_VISITS` .

Table 6.5 — Dashboard UI test cases

Test ID	Steps	Expected result	Actual	Status
TC-UI-01	Open the app with no filters	Six KPIs: 10,000 · 2,889 · $82,818,797 · 3.83 / 5 · 12.0% · 0.97 days	[record]	Not Executed
TC-UI-02	Visit Type = Inpatient	Total Visits 1,261; Avg LOS 7.69 days; room cost appears in breakdown	[record]	Not Executed
TC-UI-03	Visit Type = Emergency	Total Visits 1,465 (matches TC-APP-FLT-01)	[record]	Not Executed
TC-UI-04	Date Range 2025-01-01 → 2025-12-31	Total Visits 3,812 (matches TC-APP-FLT-02)	[record]	Not Executed
TC-UI-05	Pick only a start date in Date Range	No error; data unfiltered by date until the second date is chosen	[record]	Not Executed
TC-UI-06	Facility = City General Hospital	Total Visits 1,039; facility table shows one row	[record]	Not Executed
TC-UI-07	Specialty list	Shows 10 specialties; check whether "Ent" appears (expected defect D-02)	[record]	Not Executed
TC-UI-08	Combine filters to produce zero rows (e.g., narrow date + facility + specialty)	KPIs show 0 / 0.00; no exception; charts empty	[record]	Not Executed
TC-UI-09	Hover over any chart mark	Tooltip shows category and value	[record]	Not Executed
TC-UI-10	Sort the doctor leaderboard by AVG_SATISFACTION	Table re-sorts	[record]	Not Executed
TC-UI-11	Count doctor leaderboard rows (no filters)	Expected 60; replica predicts 59 (defect D-01)	[record]	Not Executed
TC-UI-12	Reload within 10 minutes	Second load faster (cache `ttl=600`)	[record]	Not Executed

6.7 Query Performance and Resource Usage

Measured (local replica, single run, container CPU; NOT Snowflake):

Stage	Elapsed
Load 6 CSVs into RAW (pandas → SQLite)	≈ 0.15 s
Build STAGING + CURATED (12 CTAS)	≈ 0.08 s
Query all 10 MART views	≈ 0.05 s

These numbers show only that the logic is light at this volume (10,000 fact rows). They are not evidence of Snowflake warehouse performance.

To measure in Snowflake (Not Executed): 1. Use warehouse `COMPUTE_WH` at X-Small. Optionally run `ALTER SESSION SET USE_CACHED_RESULT = FALSE;` to avoid result-cache hits. 2. Run `query.sql` sections and `tests/snowflake_validation.sql`. 3. Run the `TC-SF-PERF` query and record `TOTAL_ELAPSED_TIME`, `BYTES_SCANNED` and `ROWS_PRODUCED` per statement. 4. In Snowsight, open the *Query Profile* of `FACT_PATIENT_VISITS` creation and of `VW_DOCTOR_ANALYTICS` to inspect join and aggregate operators. 5. For the app, note first-load time (cold cache) and filtered-interaction time, and record credit usage from *Admin → Cost management*.

Expected behaviour (analytical, not measured): the app performs one full read of a 10,000 × 32 table, then filters in memory, so interactions do not query Snowflake again until the 600-second cache expires. Memory use grows linearly with fact rows, which is the main scalability limit (Phase 8).

6.8 Defects, Limitations and Corrective Actions

Table 6.6 — Defect log

ID	Severity	Found by	Description	Corrective action (proposed)	Status
D-01	Medium	TC-MART-01	`VW_DOCTOR_ANALYTICS` and the app leaderboard group by name + specialty + department, merging D018 and D020 ("Dr. Ananya Reddy", Orthopedics, Specialty Care): 338 visits shown as one doctor	Add `DOCTOR_ID` to `SELECT` / `GROUP BY` in the view and to `groupby([...])` in the app	Open
D-02	Low	TC-TRF-STD-01	`INITCAP` turns acronym `ENT` into `Ent`	`CASE WHEN UPPER(TRIM(SPECIALTY))='ENT' THEN 'ENT' ELSE INITCAP(TRIM(SPECIALTY)) END`, or drop INITCAP for already-clean columns	Open
D-03	Low	TC-TRF-STD-02	`INITCAP` turns `Follow-up` into `Follow-Up`	`INITCAP(TRIM(VISIT_TYPE), ' ')` (space-only delimiter)	Open
D-04	Medium (data)	TC-DQ-BR-01	All 1,578 "Self Pay" visits have insurance coverage (≈72.7%)	Confirm the source semantics; if Self Pay means uninsured, correct upstream or add a STAGING rule plus a DQ alert	Open: needs data owner
D-05	Low (data)	TC-DQ-BR-02	653 Emergency visits are "Admitted" with LOS = 0 and no room cost	Confirm the definition (observation stay?); document or derive LOS	Open: needs data owner

ID	Severity	Found by	Description	Corrective action (proposed)	Status
D-06	Low	Code review	`VW_PATIENT_VISIT_TYPE_ANALYSIS` duplicates `VW_VISIT_TYPE_DISTRIBUTION` (only sort differs)	Drop one view	Open
D-07	Low	Code review	`DIM_DATE` is never joined; weekday and weekend analysis unavailable	Join on `VISIT_DATE = DATE` in a view; add a weekend chart	Open
D-08	Info	Code review	App labels costs with `$`, but the source currency is unspecified (Indian cities)	Confirm currency; change labels	Open
D-09	Info	Code review	No `COPY INTO` in `query.sql`; loading is not reproducible from the repository	Add the load commands from Phase 5 §5.1.2	Open

Limitations of this test phase: Snowflake and UI tests are not executed. The business-rule checks rest on assumed semantics. No concurrency or large-volume tests were run.